Classificação de imagens do CIFAR-10 com redes convolucionais (CNN) e perceptrons multicamadas (MLP)

gbonavina

7 de novembro de 2026

Resumo

Este relatório avalia 13 redes neurais na classificação das 10 classes do CIFAR-10: nove redes convolucionais (CNN), que variam profundidade, largura, tamanho do kernel e hiperparâmetros de treino, e quatro perceptrons multicamadas (MLP). A melhor CNN, com quatro camadas convolucionais e 1.686.794 parâmetros, atingiu 91,14 % de acurácia no conjunto de teste. A segunda melhor (cinco camadas convolucionais) atingiu 89,60 %; a diferença de 1,54 pontos é estatisticamente significativa (teste de McNemar, $p \approx 2 \times 10^{-8}$). O melhor MLP, mesmo treinado por 400 épocas, chegou a 69,82 % com 3.844.682 parâmetros: 21 pontos abaixo da melhor CNN, usando 2,3 vezes mais parâmetros.

1 Introdução

O CIFAR-10 é um conjunto de imagens coloridas pequenas, muito usado para comparar arquiteturas de redes neurais. Este trabalho tem três objetivos:

1. avaliar diferentes topologias e hiperparâmetros de CNN e de MLP;
2. comparar os dois melhores modelos por meio de suas matrizes de confusão e de um teste estatístico pareado;
3. comparar a melhor CNN com o melhor MLP em acurácia e número de parâmetros.

2 Conjunto de dados

O CIFAR-10 tem 60.000 imagens RGB de 32×32 pixels, divididas igualmente em 10 classes: *airplane*, *automobile*, *bird*, *cat*, *deer*, *dog*, *frog*, *horse*, *ship* e *truck*. A divisão oficial tem 50.000 imagens de treino e 10.000 de teste (1.000 por classe). A Figura 1 mostra um exemplo de cada classe.



Figura 1: Um exemplo de cada classe do CIFAR-10, ampliado três vezes sem suavização. A baixa resolução torna algumas classes (como *cat* e *dog*) difíceis de distinguir até para humanos.

3 Metodologia

3.1 Divisão dos dados

As 50.000 imagens de treino oficiais foram embaralhadas com semente fixa e divididas em 45.000 para treino e 5.000 para validação. As 10.000 imagens de teste oficiais foram mantidas separadas. A mesma divisão foi usada em todos os modelos, o que permite comparações pareadas.

- **Treino** (45.000): ajuste dos pesos.
- **Validação** (5.000): curvas de aprendizado e escolha dos melhores modelos (*ranking*).
- **Teste** (10.000): somente para os resultados finais. Nenhuma decisão (escolha de modelo, parada de treino) usa o conjunto de teste.

Em todos os casos foi usado o modelo da última época, sem parada antecipada.

3.2 Pré-processamento e aumento de dados

Os pixels foram escalados para $[0, 1]$ e padronizados por canal com a média (0,4914; 0,4822; 0,4465) e o desvio-padrão (0,2470; 0,2435; 0,2616) do CIFAR-10. Durante o treino (e apenas no treino) foi aplicado aumento de dados em cada imagem: espelhamento horizontal com probabilidade 0,5 e recorte aleatório de 32×32 após *padding* de 4 pixels, o que desloca a imagem em até ± 4 pixels. Todos os modelos usaram aumento de dados, exceto `cnn_4conv_noaug`, que serve para medir seu efeito.

3.3 Arquiteturas

CNN. Cada camada convolucional é seguida de *batch normalization*, ReLU e *max pooling* 2×2. O número de canais começa em C (largura base) e dobra a cada camada; o *padding* mantém o tamanho espacial antes do *pooling*. Após as convoluções, um *global average pooling* reduz cada mapa a um valor, seguido de um classificador Linear$(C_L \to C_L/2) \to$ ReLU $\to$ Dropout $\to$ Linear$(C_L/2 \to 10)$, em que C_L é o número de canais da última camada.

MLP. A imagem é achatada em um vetor de 3.072 valores ($3 \times 32 \times 32$), seguido de L camadas ocultas Linear $\to$ ReLU $\to$ Dropout e de uma camada de saída com 10 neurônios.

As duas redes devolvem *logits*, usados com a função de perda de entropia cruzada. A Tabela 1 lista as 13 configurações avaliadas.

Tabela 1: Configurações avaliadas. Salvo indicação, as CNNs usam kernel 3×3, largura base $C = 32$ e *dropout* 0,5; os MLPs usam *dropout* 0,5.

Modelo	Descrição	Parâmetros
`cnn_2conv`	2 camadas conv. (32, 64 canais)	21.994
`cnn_3conv`	3 camadas conv. (32 a 128 canais)	102.602
`cnn_4conv`	4 camadas conv. (32 a 256 canais)	423.562
`cnn_5conv`	5 camadas conv. (32 a 512 canais)	1.704.458
`cnn_3conv_k5`	3 camadas conv., kernel 5×5	267.978
`cnn_4conv_w64`	4 camadas conv., largura base $C = 64$ (64 a 512)	1.686.794
`cnn_4conv_nodrop`	como `cnn_4conv`, sem *dropout*	423.562
`cnn_4conv_adamw`	como `cnn_4conv`, otimizador AdamW	423.562
`cnn_4conv_noaug`	como `cnn_4conv`, sem aumento de dados	423.562
`mlp_1x512`	1 camada oculta (512)	1.578.506
`mlp_3_1024-256`	3 camadas ocultas (1.024, 512, 256)	3.805.450
`mlp_2x2048`	2 camadas ocultas (2.048, 2.048)	10.510.346
`mlp_5_1024-64`	5 camadas ocultas (1.024 a 64), *dropout* 0,2	3.844.682

3.4 Treinamento

Todos os modelos foram treinados com *batch* de 128 imagens e política de taxa de aprendizado *one-cycle*, que sobe até a taxa máxima e depois decai até quase zero ao longo de todo o treino. O otimizador padrão foi SGD com *momentum* de Nesterov (0,9) e *weight decay* de 5×10^{-4}, com taxa máxima de 0,05 para as CNNs e 0,01 para os MLPs. O modelo `cnn_4conv_adamw` usou AdamW com taxa máxima de 2×10^{-3} e *weight decay* de 0,05.

As CNNs foram treinadas por 90 épocas. Os MLPs foram treinados por 400 épocas, porque com 90 épocas a perda de validação ainda caía de forma clara (Seção 4.4). Assim, cada arquitetura foi treinada até próximo da convergência. Os experimentos rodaram em uma GPU NVIDIA RTX A6000 com PyTorch.

3.5 Comparação estatística

Para comparar dois modelos no mesmo conjunto de teste, foi usado o teste exato de McNemar. Sejam b o número de imagens que só o modelo A acerta e c o número que só o modelo B acerta. Sob a hipótese nula de desempenho igual, $\min(b, c)$ segue uma distribuição binomial com $n = b + c$ e $p = 0{,}5$. Também foi calculado um intervalo de confiança de 95 % para a diferença de acurácia, por *bootstrap* pareado com 2.000 reamostragens.

4 Resultados

4.1 Comparação das topologias

A Tabela 2 mostra os 13 modelos ordenados pela acurácia de validação.

Tabela 2: Resultados ordenados pela acurácia de validação. A acurácia de treino foi medida durante a época, com aumento de dados e *dropout* ativos, e por isso pode ficar abaixo da de validação.

#	Modelo	Tipo	Parâmetros	Épocas	Val. (%)	Teste (%)	Treino (%)	Tempo (s)
1	`cnn_4conv_w64`	CNN	1.686.794	90	**91,30**	**91,14**	99,03	302
2	`cnn_5conv`	CNN	1.704.458	90	91,04	89,60	97,96	346
3	`cnn_4conv`	CNN	423.562	90	90,42	89,54	95,65	316
4	`cnn_4conv_nodrop`	CNN	423.562	90	90,04	89,41	97,24	295
5	`cnn_4conv_adamw`	CNN	423.562	90	89,86	89,05	96,94	296
6	`cnn_3conv_k5`	CNN	267.978	90	87,12	86,83	90,95	179
7	`cnn_3conv`	CNN	102.602	90	85,68	85,54	86,37	281
8	`cnn_4conv_noaug`	CNN	423.562	90	84,02	83,75	99,99	269
9	`mlp_5_1024-64`	MLP	3.844.682	400	71,64	69,82	72,56	1032
10	`cnn_2conv`	CNN	21.994	90	70,46	69,23	64,00	251
11	`mlp_2x2048`	MLP	10.510.346	400	68,56	68,34	67,14	808
12	`mlp_3_1024-256`	MLP	3.805.450	400	65,56	64,96	59,34	831
13	`mlp_1x512`	MLP	1.578.506	400	59,94	59,94	54,04	608

Os principais efeitos observados nas CNNs foram:

- **Profundidade.** Mais camadas melhoram muito a acurácia até quatro camadas (de 69,2 % com duas para 89,5 % com quatro). A quinta camada praticamente não ajuda (89,6 %) e quadruplica o número de parâmetros.
- **Largura.** Dobrar a largura da rede de quatro camadas (`cnn_4conv_w64`) deu o melhor resultado (91,1 %). Com quase o mesmo número de parâmetros, a rede de 4 camadas mais larga superou a de 5 camadas mais estreita.

- **Kernel.** O kernel 5×5 melhorou a rede de três camadas (85,5 % para 86,8 %), ao custo de 2,6 vezes mais parâmetros.
- ***Dropout* e otimizador.** Remover o *dropout* ou trocar SGD por AdamW mudou o resultado em menos de 0,5 ponto: com aumento de dados e *batch normalization*, esses hiperparâmetros têm pouco efeito.
- **Aumento de dados.** Foi o fator de treino mais importante. Sem ele, a mesma rede caiu de 89,5 % para 83,8 %.

A Figura 2 mostra as curvas de perda. O sobreajuste de `cnn_4conv_noaug` é claro: a perda de treino vai a praticamente zero (99,99 % de acurácia de treino), enquanto a perda de validação estaciona em torno de 0,6 a 0,8, com muita oscilação. Nas demais CNNs, as duas curvas descem juntas por mais tempo e a perda de validação termina entre 0,30 e 0,40.

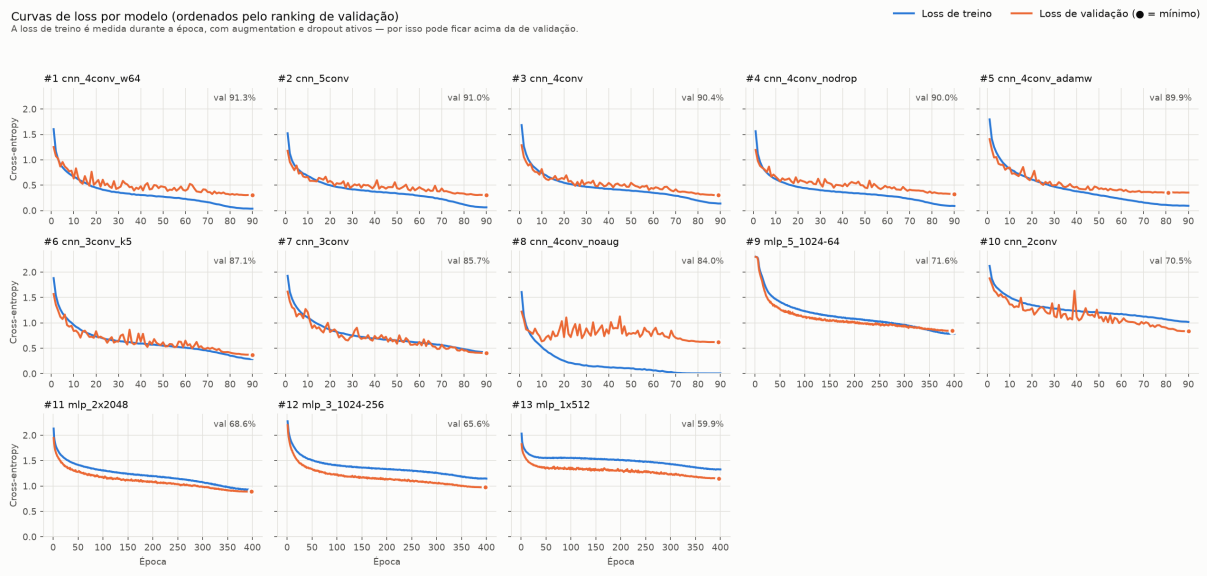


Figura 2: Perda de treino e de validação por época, para cada modelo, na ordem do *ranking*. O ponto marca o mínimo da perda de validação. Os MLPs foram treinados por 400 épocas e as CNNs por 90, por isso o eixo horizontal varia.

4.2 Os dois melhores modelos

Os dois melhores modelos na validação foram `cnn_4conv_w64` e `cnn_5conv`. A Figura 3 mostra suas matrizes de confusão no conjunto de teste.

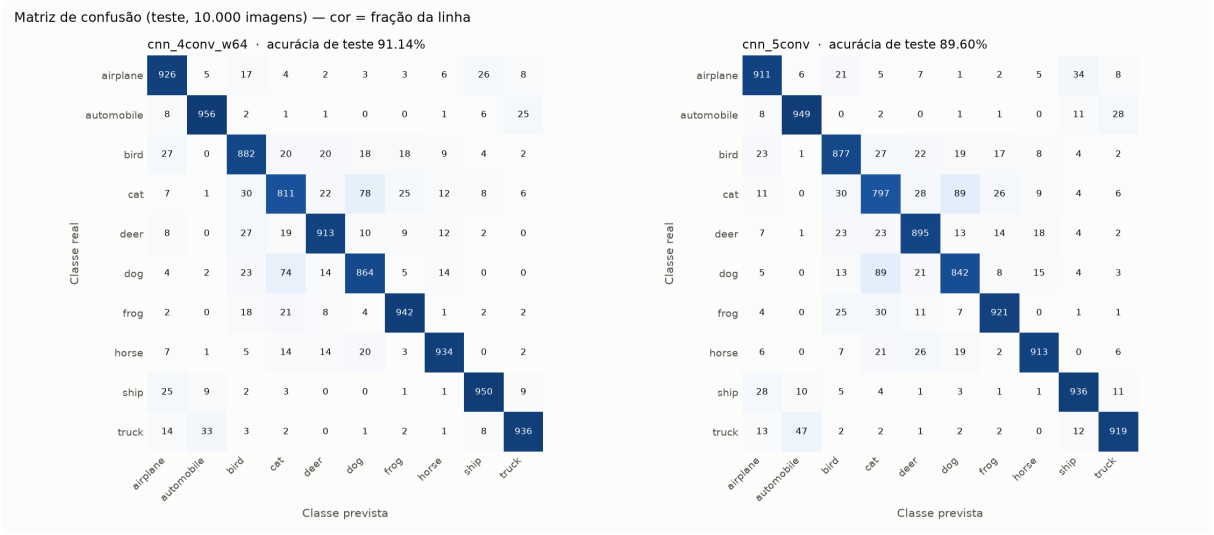


Figura 3: Matrizes de confusão dos dois melhores modelos no conjunto de teste (10.000 imagens, 1.000 por classe). Os números são contagens; a cor indica a fração de cada linha.

Os dois modelos erram de forma muito parecida. A maior confusão é entre *cat* e *dog*, nos dois sentidos: 78 gatos classificados como cachorro e 74 cachorros como gato em `cnn_4conv_w64`, contra 89 e 89 em `cnn_5conv`. Em seguida vem *truck* classificado como *automobile* (33 e 47 casos). Essas são classes visualmente parecidas, ainda mais em 32×32 pixels. A classe mais difícil para os dois modelos é *cat* (81,1 % e 79,7 %), e as mais fáceis são *automobile* e *ship* (acima de 93 %).

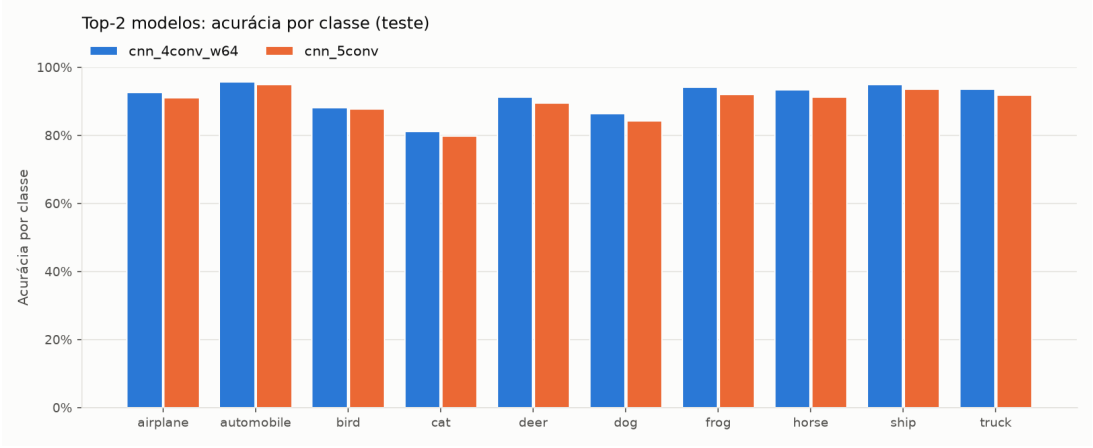


Figura 4: Acurácia por classe dos dois melhores modelos no conjunto de teste.

Diferença entre os modelos. A Tabela 3 resume a comparação pareada. Os dois modelos acertam as mesmas 8.663 imagens e erram as mesmas 589; eles discordam em 748 imagens. Dessas, `cnn_4conv_w64` acerta 451 e `cnn_5conv` acerta 297.

Tabela 3: Comparação pareada dos dois melhores modelos no conjunto de teste.

Acurácia de `cnn_4conv_w64`	91,14 %
Acurácia de `cnn_5conv`	89,60 %
Diferença	+1,54 ponto percentual
IC 95 % (*bootstrap*)	[+1,00; +2,06]
Só `cnn_4conv_w64` acerta (b)	451
Só `cnn_5conv` acerta (c)	297
Teste exato de McNemar	$p = 1{,}98 \times 10^{-8}$

Com $p < 0{,}05$ e um intervalo de confiança que não contém o zero, a diferença de 1,54 ponto é estatisticamente significativa. Ela é pequena em termos práticos, mas consistente: `cnn_4conv_w64` é melhor em todas as 10 classes (Figura 4), com as maiores vantagens em *dog*, *frog* e *horse* (cerca de 2 pontos cada). Como os dois modelos têm quase o mesmo número de parâmetros (1,69 M e 1,70 M), o resultado indica que, neste orçamento de parâmetros, uma rede mais larga e menos profunda funciona melhor do que uma mais profunda e mais estreita.

4.3 CNN versus MLP

A Tabela 4 compara a melhor CNN com o melhor MLP.

Tabela 4: Melhor CNN versus melhor MLP.

	CNN (`cnn_4conv_w64`)	MLP (`mlp_5_1024-64`)
Acurácia de teste	91,14 %	69,82 %
Parâmetros	1.686.794	3.844.682
Épocas	90	400
Tempo de treino	302 s	1.032 s

A CNN acerta 21,3 pontos a mais usando 2,3 vezes menos parâmetros, menos épocas e menos tempo de treino (McNemar, $p < 10^{-10}$). A Figura 5 mostra que essa vantagem vale para todos os modelos: qualquer CNN com três ou mais camadas (102.602 parâmetros ou mais) supera todos os MLPs, inclusive o de 10,5 milhões de parâmetros. O melhor MLP só supera a menor CNN (`cnn_2conv`, 69,2 %), e para isso precisa de 175 vezes mais parâmetros.

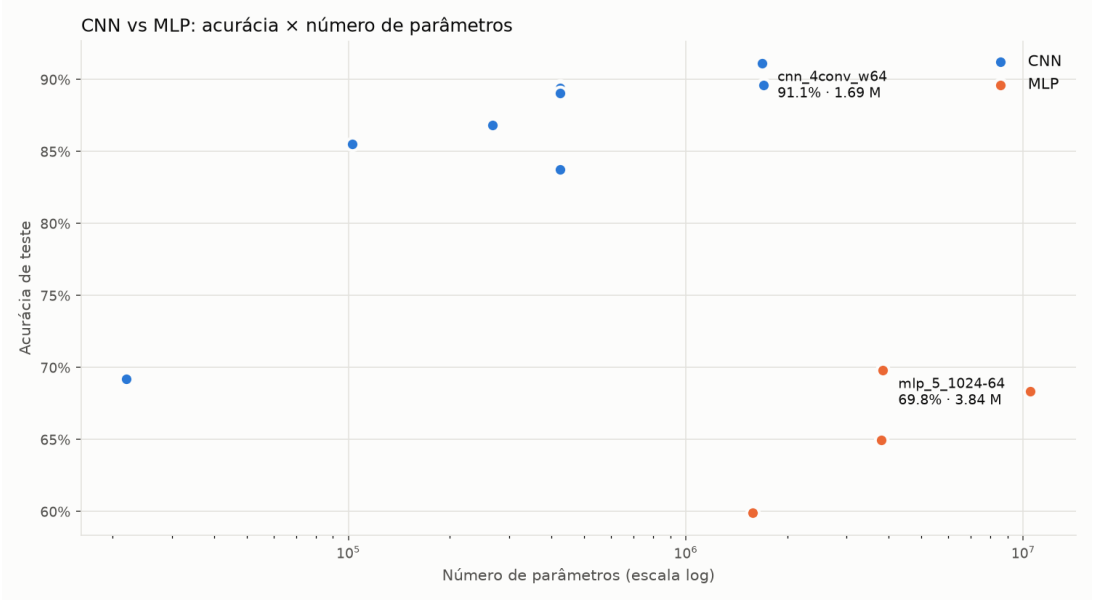


Figura 5: Acurácia de teste em função do número de parâmetros (escala logarítmica).

A diferença vem da arquitetura. Na CNN, os mesmos filtros pequenos são aplicados em todas as posições da imagem: a rede reconhece um padrão (uma borda, um olho, uma roda) onde quer que ele apareça, e o *pooling* torna a resposta tolerante a pequenos deslocamentos. O MLP achata a imagem e liga cada pixel a pesos próprios, perdendo a estrutura espacial; um objeto deslocado alguns pixels vira, para ele, uma entrada totalmente diferente. Por isso o MLP precisa de muito mais parâmetros e exemplos para aprender a mesma coisa, e o aumento de dados por deslocamento, que ajuda muito a CNN, torna o aprendizado do MLP mais lento.

As confusões do MLP são as mesmas da CNN, mas muito mais frequentes: 230 cachorros classificados como gato, 153 gatos como cachorro e 119 automóveis como caminhão. A acurácia por classe do MLP vai de 54,8 % (*dog*) a 83,9 % (*ship*).

4.4 Efeito do número de épocas

Todos os modelos foram treinados primeiro por 30 épocas e depois por 90. Os MLPs também foram treinados por 400 épocas. A Tabela 5 mostra a acurácia de teste em cada caso.

Tabela 5: Acurácia de teste (%) em função do número de épocas de treino.

Modelo	30 épocas	90 épocas	400 épocas
`cnn_4conv_w64`	90,08	91,14	–
`cnn_5conv`	87,74	89,60	–
`cnn_4conv`	87,31	89,54	–
`cnn_4conv_nodrop`	87,95	89,41	–
`cnn_4conv_adamw`	87,09	89,05	–
`cnn_3conv_k5`	84,93	86,83	–
`cnn_3conv`	81,76	85,54	–
`cnn_4conv_noaug`	83,28	83,75	–
`cnn_2conv`	64,75	69,23	–
`mlp_5_1024-64`	55,47	62,50	69,82
`mlp_2x2048`	55,34	61,21	68,34
`mlp_3_1024-256`	52,47	57,97	64,96
`mlp_1x512`	53,99	56,77	59,94

As CNNs grandes ganharam de 1 a 2 pontos de 30 para 90 épocas, e suas perdas de validação se estabilizam no fim do treino. A exceção é `cnn_4conv_noaug`, que ganhou só 0,5 ponto: sem aumento de dados, as épocas extras servem apenas para decorar o conjunto de treino. Os MLPs ganharam mais (de cerca de 3 a 7 pontos a cada etapa), porque estavam subajustados: sua perda de treino ficava acima da de validação. Com 400 épocas, o melhor MLP (`mlp_5_1024-64`) parece ter convergido, já que sua perda de treino cruza a de validação no fim. O `mlp_1x512` se estabiliza em torno de 60 %, um limite da sua capacidade. A seleção dos dois melhores modelos e a conclusão da comparação CNN versus MLP são as mesmas com 30, 90 ou 400 épocas.

5 Conclusão

- A melhor rede foi `cnn_4conv_w64`, uma CNN de quatro camadas convolucionais com largura base 64, com 91,14 % de acurácia de teste e 1,7 milhão de parâmetros.
- Ela supera a segunda melhor (`cnn_5conv`) por 1,54 ponto, com diferença estatisticamente significativa ($p \approx 2 \times 10^{-8}$) e vantagem em todas as classes. Os dois modelos confundem principalmente *cat* e *dog*.
- Profundidade até quatro camadas, largura e aumento de dados foram os fatores mais importantes. *Dropout* e a escolha do otimizador tiveram pouco efeito.

- O melhor MLP chegou a 69,82 % mesmo com 400 épocas, 21 pontos abaixo da melhor CNN e com 2,3 vezes mais parâmetros. Para imagens, o compartilhamento de pesos e a estrutura espacial das convoluções são muito mais eficientes do que camadas totalmente conectadas.

Limitações. Cada configuração foi treinada uma única vez, com uma semente; diferenças pequenas entre modelos do meio do *ranking* (como entre `cnn_4conv`, `cnn_4conv_nodrop` e `cnn_4conv_adamw`) podem estar dentro da variação entre execuções. Os MLPs foram avaliados apenas com aumento de dados; versões sem o recorte aleatório poderiam convergir mais rápido.

Reprodutibilidade. O código está em `src/`. Os resultados deste relatório foram gerados com:

```
uv run python src/run_experiments.py --epochs 90 --out results_90ep
uv run python src/run_experiments.py --epochs 400 --out results_mlp400 \
    --only mlp_1x512 mlp_3_1024-256 mlp_2x2048 mlp_5_1024-64
uv run python src/run_experiments.py --merge results_90ep results_mlp400 \
    --out results_final
```